

DELETE: borrar filas o desactivarlas

En Excel, borrar una fila es algo cotidiano. Seleccionas, borras, listo. En una base de datos de negocio, borrar una fila tiene consecuencias que van mucho más allá de esa fila. Si eliminas un cliente, no solo desaparece su nombre y correo: desaparece todo su historial de pedidos, todas las relaciones que tenía con otras tablas, toda la trazabilidad de lo que compró. El motor no te avisa ni te pide confirmación. Simplemente lo hace.

Por eso, en la práctica, los equipos que trabajan con bases de datos de negocio casi nunca usan **DELETE** para datos operativos. Usan otra estrategia.

DELETE físico: cómo funciona y cuándo usarlo

DELETE elimina filas de una tabla de forma permanente. La estructura es la misma que ya conoces de **UPDATE**: el comando, la tabla y un **WHERE** que identifica exactamente qué eliminar.

```
SQL
DELETE FROM clientes
WHERE cliente_id = 99;
```

Esta instrucción borra la fila del cliente con ID 99. Una vez ejecutada, esa fila no existe más en la tabla.

La misma regla de oro del **UPDATE** aplica aquí con consecuencias todavía más graves:

```
SQL
-- ✗ Esto borra TODOS los clientes de la tabla
DELETE FROM clientes;
```

Sin **WHERE**, el motor elimina todas las filas sin error ni advertencia. Y a diferencia de un **UPDATE** que puedes corregir si sabes los valores anteriores, un **DELETE** sin respaldo no tiene vuelta atrás.

Para confirmar qué filas se eliminaron, puedes usar **RETURNING**:

```
SQL
DELETE FROM detalle_pedidos

WHERE pedido_id = 100

RETURNING *;
```

El motor ejecuta el borrado y devuelve las filas que acaba de eliminar. Es la última oportunidad de ver esos datos antes de que desaparezcan.

El problema real del DELETE en datos de negocio

Imagina este escenario: un cliente llama para pedir que lo des de baja. Ejecutas **DELETE FROM clientes WHERE cliente_id = 5**. Perfecto, el cliente ya no está.

Pero ahora tienes un problema: todos los pedidos que ese cliente hizo siguen en la tabla de pedidos con su **cliente_id**. Esas filas quedaron huérfanas, apuntando a un cliente que ya no existe. Los reportes históricos de ventas ya no cierran. La integridad referencial que tanto trabajo costó construir quedó rota.

DELETE tiene sentido para limpiar datos temporales, registros de prueba o errores de carga. Para datos operativos reales, la alternativa es el borrado lógico.

Borrado lógico: desactivar en lugar de eliminar

El borrado lógico no borra nada. En cambio, marca la fila como inactiva usando el campo **activo** que ya existe en las tablas de Tienda Latam. La fila sigue en la base de datos, pero se trata como si no existiera para las consultas normales.

sql

```
SQL
-- En vez de DELETE, usas UPDATE:

UPDATE clientes

SET activo = FALSE

WHERE cliente_id = 5;
```

El cliente sigue en la tabla. Su historial de pedidos sigue intacto. La integridad referencial no se rompe. La única diferencia es que **activo** ahora es **FALSE**.

A partir de ese momento, la forma de consultarlo cambia:

```
SQL
-- Ver solo clientes activos:

SELECT * FROM clientes

WHERE activo = TRUE;

-- Ver solo clientes inactivos:

SELECT * FROM clientes

WHERE activo = FALSE;
```

Y si ese cliente vuelve a hacer negocios contigo, simplemente lo reactivás:

```
SQL
UPDATE clientes

SET activo = TRUE

WHERE cliente_id = 5;
```

No necesitas volver a crearlo, no pierdes su historial, no rompes ninguna relación. El dato siempre estuvo ahí.

Cuándo usar cada uno

Situación	Qué usar
Limpiar datos de prueba o temporales	DELETE físico
Dar de baja un cliente, empleado o producto	Borrado lógico
Eliminar un pedido con error de carga	DELETE físico
Cancelar un pedido real que ya tiene historial	Borrado lógico
Corregir registros duplicados por error	DELETE físico
Desactivar una sucursal que cerró	Borrado lógico

La pregunta que guía la decisión es siempre la misma: ¿este dato tiene historial o relaciones con otras tablas que necesito conservar? Si la respuesta es sí, el borrado lógico es la opción correcta.

La regla de oro, ahora con DELETE

Ya la viste con UPDATE. Con DELETE aplica con la misma lógica y el mismo hábito:

```
SQL
-- Primero verifica qué filas vas a borrar:

SELECT * FROM clientes

WHERE cliente_id = 99;

-- Si el resultado es correcto, ejecutas el DELETE:
```

```
DELETE FROM clientes  
WHERE cliente_id = 99;
```

Si el **SELECT** devuelve las filas correctas, el **DELETE** con el mismo **WHERE** va a eliminar exactamente esas. Ni una más.

Con **INSERT**, **UPDATE** y **DELETE** dominados, tienes todo lo que necesitas para gestionar los datos de una tabla. El siguiente paso es aprender a cruzar información entre varias tablas en una sola consulta, que es donde SQL empieza a responder las preguntas de negocio más interesantes.